

Furqan Reasoning-as-a-Skill (RaaS)

Architecture Document v1.0

Project: Al-Furqan Reasoning Engine → Universal AI Skill **Version:** 1.0 — Draft **Date:** March 21, 2026
Authors: R&D/Innovation Lab **Status:** Proposed

1. Vision

1.1 The Problem

Every LLM hallucinates. Every LLM lacks source verification. No LLM can **prove** its reasoning is consistent.

Al-Furqan already solves this — but only for direct API users.

1.2 The Goal

Turn Al-Furqan's Reasoning Engine into a universal Skill that any AI agent — Claude, GPT, Gemini, local Ollama, or any MCP-compatible agent — can invoke as a tool.

The agent says: *"I need to verify this claim against trusted sources."* The Skill does the heavy lifting: retrieval → reasoning → verification → sourced response.

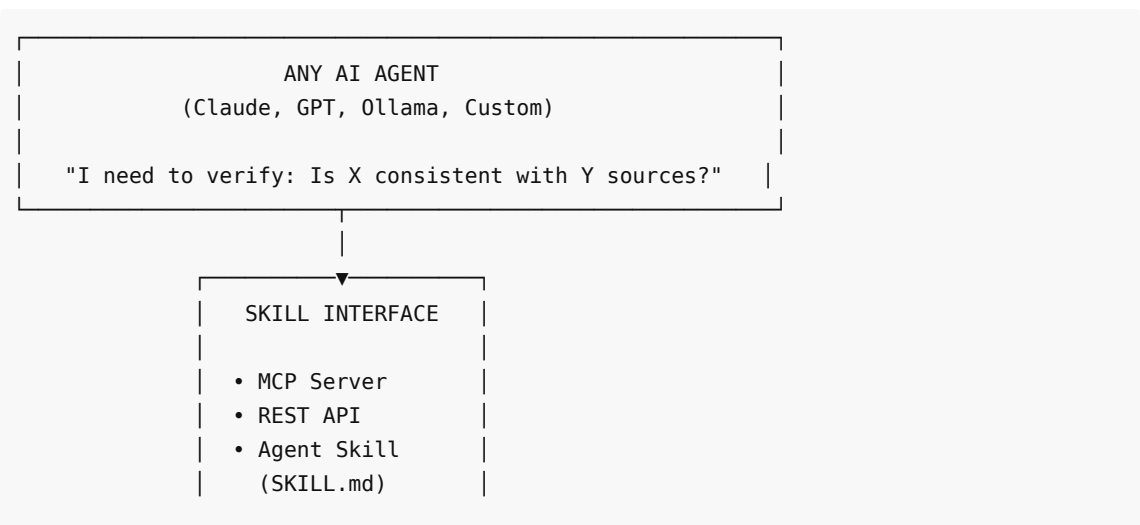
1.3 Key Principle: Domain-Agnostic Core

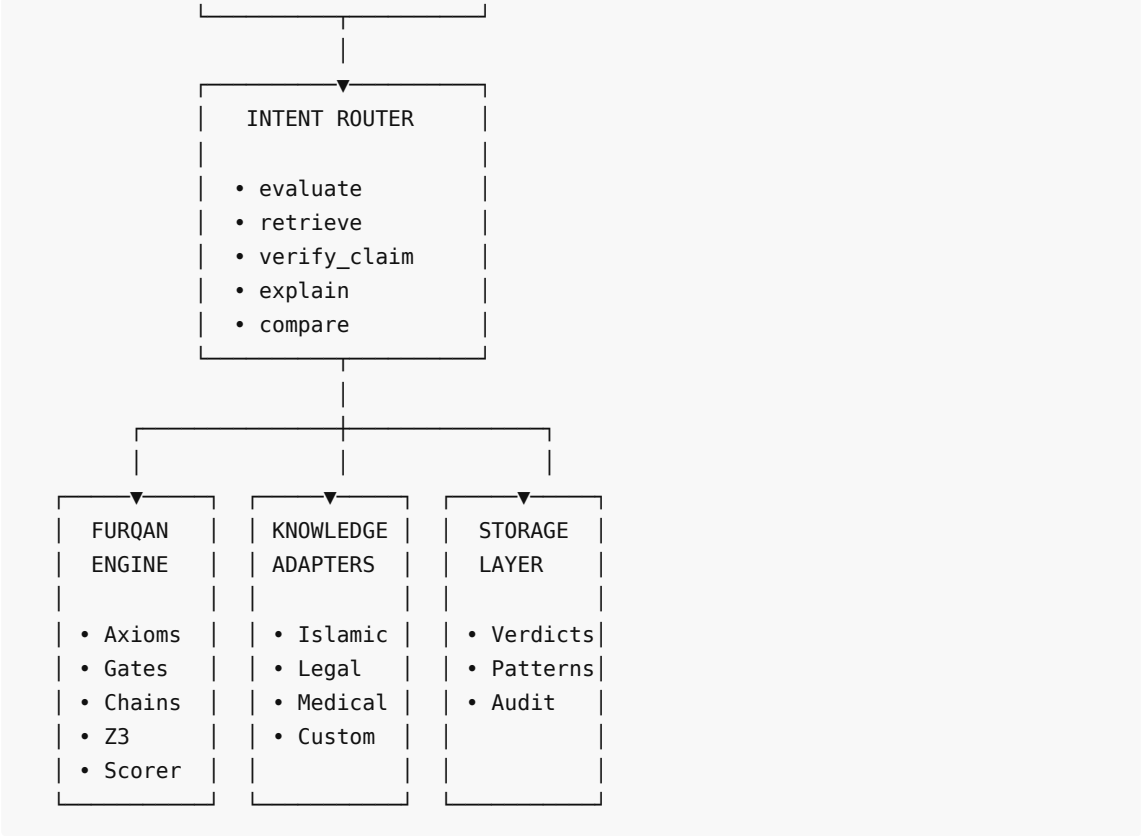
The engine is **NOT** tied to any single domain. It's a general-purpose axiom-anchored reasoning system.

- **Islamic Knowledge** = First vertical (proven, tested, deployed)
- **Legal, Medical, Educational, Scientific** = Future verticals
- **Same engine, same gates, same verification** — only the Knowledge Adapter changes

2. Architecture Overview

2.1 High-Level Design





2.2 The Three Access Methods

Method	Who Uses It	Protocol	Use Case
MCP Server	Any MCP-compatible agent	Model Context Protocol	Native AI agent integration
REST API	Any HTTP client	HTTP/JSON	Web apps, mobile, custom agents
Agent Skill	OpenClaw / Claude Code agents	SKILL.md + local CLI	Direct skill invocation

All three hit the **same engine** — different doors, same house.

3. Skill Interface Design

3.1 SKILL.md (Agent Skill Definition)

```
name: furqan-reasoning
description: >
  General-purpose axiom-anchored reasoning engine.
  Evaluates claims, ideas, and systems against verified
  knowledge sources with formal verification (Z3).
  Domain-agnostic: supports Islamic, legal, medical,
  educational, and custom knowledge bases.
```

```

commands:
  evaluate:
    description: Full evaluation through all 4 gates
    params:
      question: string (required)
      domain: string (default: "islamic")
      depth: enum [quick, standard, deep]
      language: string (default: "ar")

  verify:
    description: Verify a specific claim against sources
    params:
      claim: string (required)
      domain: string (default: "islamic")

  retrieve:
    description: Retrieve relevant sources without evaluation
    params:
      query: string (required)
      domain: string (default: "islamic")
      limit: int (default: 10)

  explain:
    description: Explain a topic with sourced references
    params:
      topic: string (required)
      domain: string (default: "islamic")
      depth: enum [brief, detailed]

  compare:
    description: Compare multiple viewpoints on a topic
    params:
      topic: string (required)
      perspectives: list[string]
      domain: string (default: "islamic")

domains:
  description: List available knowledge domains

```

3.2 MCP Server Tools

```

{
  "tools": [
    {
      "name": "furqan_evaluate",
      "description": "Evaluate a question/claim through axiom-anchored reasoning with formal verification. Returns a verdict with gate scores, source citations, and proof trail.",
      "inputSchema": {
        "type": "object",

```

```

    "properties": {
      "question": { "type": "string", "description": "The question or claim to
evaluate" },
      "domain": { "type": "string", "default": "islamic", "description":
"Knowledge domain to use" },
      "depth": { "type": "string", "enum": ["quick", "standard", "deep"],
"default": "standard" },
      "include_sources": { "type": "boolean", "default": true }
    },
    "required": ["question"]
  },
  {
    "name": "furqan_verify",
    "description": "Verify a specific claim against trusted knowledge sources.
Returns verification status, matching sources, and confidence score.",
    "inputSchema": {
      "type": "object",
      "properties": {
        "claim": { "type": "string" },
        "domain": { "type": "string", "default": "islamic" }
      },
      "required": ["claim"]
    }
  },
  {
    "name": "furqan_retrieve",
    "description": "Search and retrieve relevant knowledge from verified
sources.",
    "inputSchema": {
      "type": "object",
      "properties": {
        "query": { "type": "string" },
        "domain": { "type": "string", "default": "islamic" },
        "limit": { "type": "integer", "default": 10 }
      },
      "required": ["query"]
    }
  },
  {
    "name": "furqan_explain",
    "description": "Get a sourced explanation of a topic from verified knowledge
bases.",
    "inputSchema": {
      "type": "object",
      "properties": {
        "topic": { "type": "string" },
        "domain": { "type": "string", "default": "islamic" },
        "depth": { "type": "string", "enum": ["brief", "detailed"], "default":
"detailed" }
      },
      "required": ["topic"]
    }
  }
}

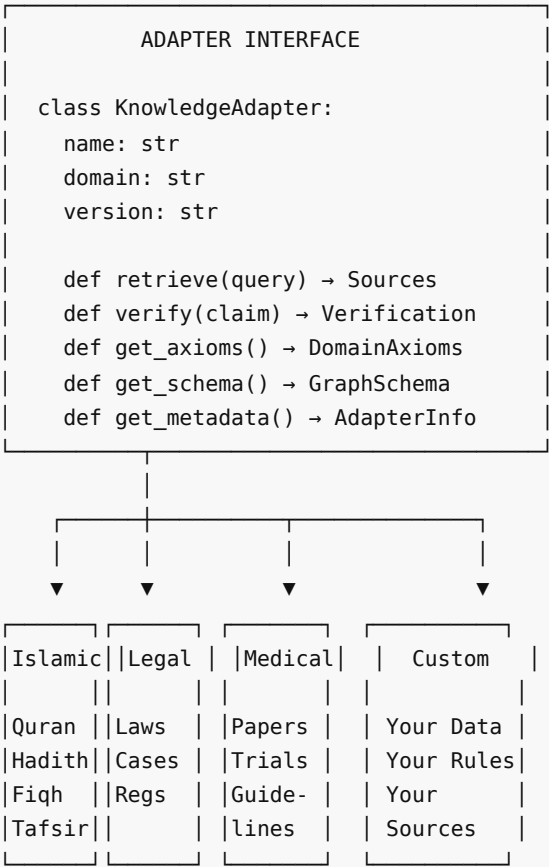
```

```
}
}
]
}
```

4. Knowledge Adapter System

4.1 The Adapter Pattern

The **core engine** (axioms, gates, chains, Z3, scorer) stays the same. The **knowledge layer** is pluggable.



4.2 Adapter Interface

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Optional

@dataclass
class Source:
    """A single verified source reference."""
    id: str
    text: str
```

```

reference: str          # e.g., "Quran 2:255" or "Civil Code Art. 147"
domain: str
confidence: float       # 0.0 - 1.0
metadata: dict = field(default_factory=dict)

```

```
@dataclass
```

```

class RetrievalResult:
    """Results from a knowledge retrieval."""
    sources: list[Source]
    query: str
    domain: str
    total_found: int
    formatted_context: str # Ready for engine consumption

```

```
@dataclass
```

```

class VerificationResult:
    """Result of verifying a claim against sources."""
    claim: str
    verified: bool
    confidence: float
    supporting_sources: list[Source]
    contradicting_sources: list[Source]
    explanation: str

```

```
@dataclass
```

```

class DomainAxioms:
    """Domain-specific axioms that extend core axioms."""
    domain: str
    axioms: list[str]          # Domain truths
    verification_rules: list[str] # How to verify in this domain
    authority_chain: list[str]   # Who/what is authoritative

```

```

class KnowledgeAdapter(ABC):
    """Base class for all knowledge domain adapters."""

```

```

@property
@abstractmethod
def name(self) -> str: ...

```

```

@property
@abstractmethod
def domain(self) -> str: ...

```

```

@property
@abstractmethod
def version(self) -> str: ...

```

```

@abstractmethod
def retrieve(self, query: str, limit: int = 10) -> RetrievalResult:
    """Retrieve relevant sources for a query."""
    ...

```

```

@abstractmethod
def verify(self, claim: str) -> VerificationResult:
    """Verify a specific claim against this domain's sources."""
    ...

@abstractmethod
def get_axioms(self) -> DomainAxioms:
    """Return domain-specific axioms and verification rules."""
    ...

def get_metadata(self) -> dict:
    """Return adapter metadata (source count, last updated, etc.)."""
    return {
        "name": self.name,
        "domain": self.domain,
        "version": self.version,
    }

```

4.3 Islamic Adapter (First Implementation)

```

class IslamicKnowledgeAdapter(KnowledgeAdapter):
    """
    First adapter – wraps existing KB layer.
    Quran (6,236 verses) + Hadith (38,016+) + Fiqh (50 rules)
    + Knowledge Graph + Scholarly chains.
    """
    name = "al-furqan-islamic"
    domain = "islamic"
    version = "1.0"

    def __init__(self, kb_retriever, knowledge_linker):
        self.retriever = kb_retriever
        self.linker = knowledge_linker

    def retrieve(self, query: str, limit: int = 10) -> RetrievalResult:
        # Wraps existing unified retriever
        results = self.retriever.retrieve(query, limit=limit)
        return RetrievalResult(
            sources=[self._to_source(r) for r in results],
            query=query,
            domain="islamic",
            total_found=len(results),
            formatted_context=self._format_context(results),
        )

    def verify(self, claim: str) -> VerificationResult:
        # Search for supporting/contradicting sources
        sources = self.retriever.retrieve(claim)
        # Use knowledge linker for scholarly chain verification
        chain = self.linker.build_chain(claim, sources)
        return VerificationResult(

```

```

        claim=claim,
        verified=chain.is_consistent,
        confidence=chain.confidence,
        supporting_sources=chain.supporting,
        contradicting_sources=chain.contradicting,
        explanation=chain.explanation,
    )

def get_axioms(self) -> DomainAxioms:
    return DomainAxioms(
        domain="islamic",
        axioms=[
            "The Quran is the primary, unaltered source of truth",
            "Authentic Hadith (Sahih/Hasan) complement Quranic guidance",
            "Scholarly consensus (Ijma') carries binding authority",
            "Analogical reasoning (Qiyas) is valid when conditions are met",
        ],
        verification_rules=[
            "Every claim must trace to Quran or authenticated Hadith",
            "Hadith grading must be verified (Sahih > Hasan > Da'if)",
            "Context (Asbab al-Nuzul) must be considered",
            "Multiple tafsir sources should be cross-referenced",
        ],
        authority_chain=[
            "Quran → Mutawatir Hadith → Ahad Hadith → Ijma' → Qiyas"
        ],
    )

```

4.4 Adding a New Domain (Example: Legal)

```

class LegalKnowledgeAdapter(KnowledgeAdapter):
    """Example: Egyptian Civil Law adapter."""
    name = "furqan-legal-eg"
    domain = "legal"
    version = "0.1"

    def retrieve(self, query: str, limit: int = 10) -> RetrievalResult:
        # Search law database, case precedents, regulatory texts
        ...

    def verify(self, claim: str) -> VerificationResult:
        # Check against specific articles, precedents
        ...

    def get_axioms(self) -> DomainAxioms:
        return DomainAxioms(
            domain="legal",
            axioms=[
                "Constitutional provisions override all other law",
                "Later legislation supersedes earlier on same subject",
                "Court of Cassation rulings set binding precedent",
            ],
        )

```



```

    ],
    verification_rules=[
        "Every legal claim must cite specific articles",
        "Amendment status must be verified (current vs repealed)",
        "Jurisdiction must be confirmed",
    ],
    authority_chain=[
        "Constitution → Law → Executive Regulation → Precedent"
    ],
)

```

5. MCP Server Architecture

5.1 Server Design



5.2 Implementation (Python + MCP SDK)

```

from mcp.server import Server
from mcp.types import Tool, TextContent

app = Server("furqan-reasoning")

# Registry of available adapters
adapters: dict[str, KnowledgeAdapter] = {}
engine: FurqanEngine = None

@app.list_tools()
async def list_tools() -> list[Tool]:
    return [
        Tool(
            name="furqan_evaluate",
            description="Evaluate a claim through axiom-anchored reasoning",
            inputSchema={...} # as defined in 3.2
        ),
        Tool(name="furqan_verify", ...),
        Tool(name="furqan_retrieve", ...),
        Tool(name="furqan_explain", ...),
        Tool(name="furqan_domains", ...),
    ]

@app.call_tool()
async def call_tool(name: str, arguments: dict):
    domain = arguments.get("domain", "islamic")
    adapter = adapters.get(domain)
    if not adapter:
        return [TextContent(text=f"Domain '{domain}' not available. Use
furqan_domains to list available domains.")]

    if name == "furqan_evaluate":
        context = adapter.retrieve(arguments["question"])
        verdict = engine.evaluate(arguments["question"], context.formatted_context)
        return [TextContent(text=verdict.to_json())]

    elif name == "furqan_verify":
        result = adapter.verify(arguments["claim"])
        return [TextContent(text=result.to_json())]

    elif name == "furqan_retrieve":
        result = adapter.retrieve(arguments["query"], arguments.get("limit", 10))
        return [TextContent(text=result.to_json())]

    elif name == "furqan_domains":
        domains = {k: v.get_metadata() for k, v in adapters.items()}
        return [TextContent(text=json.dumps(domains))]

```

6. Response Format

6.1 Evaluate Response

```
{
  "status": "success",
  "verdict": {
    "question": "Is fractional reserve banking ethical?",
    "domain": "islamic",
    "gate_scores": [
      {"gate": "Source Integrity", "score": 35, "result": "Fail", "reasoning": "..."},
      {"gate": "Structural Consistency", "score": 28, "result": "Fail", "reasoning": "..."},
      {"gate": "Mediation Zeroing", "score": 45, "result": "Fail", "reasoning": "..."},
      {"gate": "Origin Aware", "result": "Fail"}
    ],
    "total_score": 36,
    "final_judgment": "Fractional reserve banking fails all gates...",
    "sources": [
      {"ref": "Quran 2:275", "text": "...أحل الله البيع وحرم الربا...", "relevance": 0.95},
      {"ref": "Sahih Muslim 1598", "text": "...", "relevance": 0.88}
    ],
    "z3_proof": "UNSAT – internal contradiction detected in debt multiplication model",
    "consequences": {
      "short_term": [...],
      "long_term": [...]
    }
  },
  "meta": {
    "engine_version": "2.0",
    "adapter": "al-furqan-islamic",
    "processing_time_ms": 4200,
    "model_used": "claude-sonnet-4-20250514"
  }
}
```

6.2 Verify Response

```
{
  "status": "success",
  "verification": {
    "claim": "Honey is mentioned as a healing substance in the Quran",
    "verified": true,
    "confidence": 0.97,
    "supporting_sources": [
      {"ref": "Quran 16:69", "text": "...فيه شفاء للناس...", "type": "primary"}
    ],
    "contradicting_sources": []
  }
}
```

```
    "explanation": "Direct Quranic reference in Surah An-Nahl, verse 69..."
  }
}
```

7. Integration Patterns

7.1 Agent Using Furqan Skill (Example Flow)

User: "Is it okay to take out a mortgage with interest?"

Agent (Claude/GPT/etc):

1. Detects: ethical/religious question → needs verified reasoning
2. Calls: `furqan_evaluate(question="mortgage with interest", domain="islamic")`
3. Receives: Full verdict with gate scores + sources
4. Responds: Synthesizes verdict in natural language with citations

Result: Agent provides sourced, verified answer instead of hallucinating

7.2 Multi-Domain Query

User: "Is surrogacy legal and religiously permissible in Egypt?"

Agent:

1. Calls: `furqan_evaluate(question="surrogacy", domain="legal")`
→ Legal verdict: Egyptian law position
2. Calls: `furqan_evaluate(question="surrogacy", domain="islamic")`
→ Islamic verdict: Scholarly positions
3. Synthesizes both verdicts into comprehensive answer

7.3 Verification Chain

User: "Someone told me the Quran mentions 360 joints in the human body"

Agent:

1. Calls: `furqan_verify(claim="Quran mentions 360 joints", domain="islamic")`
→ verified: false (it's in Hadith, not Quran)
→ correction: "Sahih Muslim 1007 – this is a Hadith, not a Quranic verse"
2. Responds with accurate correction + proper source

8. Deployment Options

8.1 Local (Embedded)

```
# MCP config for Claude Code / OpenClaw
mcpServers:
  furqan:
    command: python
    args: ["-m", "al_furqan.mcp_server"]
```

```
env:
  FURQAN_DOMAINS: "islamic"
  FURQAN_LLM_PROVIDER: "ollama" # fully local
```

Best for: Privacy-sensitive deployments, offline use, edge devices.

8.2 Remote (Shared Server)

```
# MCP config pointing to remote server
mcpServers:
  furqan:
    url: "https://furqan.variance.com/mcp"
    transport: "sse"
    headers:
      Authorization: "Bearer ${FURQAN_API_KEY}"
```

Best for: Multi-user, teams, production apps.

8.3 Hybrid

Local engine + remote knowledge base. Engine runs on device, KB queries go to server.

9. Implementation Roadmap

Phase 1: Skill Wrapper (Week 1-2)

- ☐ Create `KnowledgeAdapter` base class
- ☐ Wrap existing KB as `IslamicKnowledgeAdapter`
- ☐ Create `SKILL.md` for agent integration
- ☐ CLI tool: `furqan evaluate "question"`
- ☐ Basic REST endpoint wrapping existing API

Phase 2: MCP Server (Week 3-4)

- ☐ Implement MCP server with stdio transport
- ☐ Register tools: `evaluate`, `verify`, `retrieve`, `explain`, `domains`
- ☐ Test with Claude Code + OpenClaw
- ☐ SSE transport for remote access
- ☐ Authentication + rate limiting for remote

Phase 3: Adapter System (Week 5-6)

- ☐ Refactor engine to accept `DomainAxioms` from adapter
- ☐ Domain router: select adapter by domain parameter
- ☐ Adapter registration + discovery
- ☐ Create adapter template + documentation
- ☐ Test with 2nd domain (legal or medical)

Phase 4: Production (Week 7-8)

- ☐ Deploy MCP server on Variiance infra
- ☐ Dashboard: usage stats, domain stats, verdict browser
- ☐ SDK packages: Python, TypeScript
- ☐ Public docs + adapter creation guide
- ☐ Performance optimization (caching, batch evaluation)

10. Security Considerations

Concern	Mitigation
Prompt injection via queries	Input sanitization (already implemented)
Unauthorized domain access	Per-domain API keys + role-based access
Knowledge base tampering	Immutable source hashes + audit log
Model manipulation	LLM as tongue, not brain — code verifies, Z3 proves
Rate abuse	Per-key rate limiting (token bucket)
Data exfiltration	No PII in knowledge base; verdict store is isolated

11. Success Metrics

Metric	Target
Source accuracy	>95% of citations are correct and verifiable
Gate consistency	Same input → same gate scores (deterministic)
Response time (quick)	<2s
Response time (standard)	<5s
Response time (deep)	<15s
Adapter onboarding	New domain live in <1 week
Agent adoption	3+ agents using Furqan as tool within 3 months

12. What Makes This Different

Feature	Typical RAG	Furqan RaaS
Source verification	✗ Approximate	✓ Exact citations with grading
Reasoning proof	✗ None	✓ Z3 formal verification
Consistency	✗ Varies by prompt	✓ Deterministic scoring
Multi-domain	✗ Single embedding space	✓ Pluggable adapters

Hallucination guard	✗ Hope for the best	✓ 4-gate survival system
Agent-native	✗ REST only	✓ MCP + Skill + REST

This document is a living draft. Updated as architecture evolves.